

A PROJECT REPORT ON

**ENHANCING SOCIAL MEDIA TEXT MODERATION THROUGH
SENTIMENT ANALYSIS AND CYBERBULLYING DETECTION**

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

**BACHELOR OF ENGINEERING
(COMPUTER ENGINEERING)**

SUBMITTED BY

TEJASVI ALUR
KHUSHI GANDHI
RAJ NATHANI
RITANSHU KOUL

Exam No: B400050609
Exam No: B400050394
Exam No: B400050561
Exam No: B400050569



DEPARTMENT OF COMPUTER ENGINEERING
PUNE INSTITUTE OF COMPUTER TECHNOLOGY
DHANKAWADI, PUNE 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

“ENHANCING SOCIAL MEDIA TEXT MODERATION THROUGH SENTIMENT ANALYSIS AND CYBERBULLYING DETECTION”

Submitted by

TEJASVI ALUR
KHUSHI GANDHI
RAJ NATHANI
RITANSHU KOUL

Exam No: B400050609
Exam No: B400050394
Exam No: B400050561
Exam No: B400050569

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. P. J. Jambhulkar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of Bachelor of Engineering (Computer Engineering).

Prof. P. J. Jambhulkar
Guide,
Department of Computer Engineering

Dr. G. V. Kale
Head,
Department of Computer Engineering

Dr. S. T. Gandhe
Principal,
Pune Institute of Computer Technology

Place: Pune
Date:

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ‘**ENHANCING SOCIAL MEDIA TEXT MODERATION THROUGH SENTIMENT ANALYSIS AND CYBERBULLYING DETECTION.**’

We would like to thank our project guide **Prof. P. J. Jambhulkar** for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.

We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.

Our special thanks to our honorable **Dr. P. T. Kulkarni**, Director, PICT for their continued support and valuable inputs during ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.

Khushi Gandhi

Raj Nathani

Ritanshu Koul

Tejasvi Alur

(B.E. Computer Engg.)

ABSTRACT

Social media platforms have become central to modern communication, yet the rise in harmful and abusive content poses serious challenges to user safety and well-being. With the overwhelming volume of user-generated posts on platforms like Twitter and Reddit, manual moderation is no longer viable. This project addresses the urgent need for automated, intelligent content moderation systems capable of identifying emotionally charged, context-sensitive, and abusive language in real time. By leveraging natural language processing and deep learning, the system aims to distinguish between benign emotional expression and harmful intent.

To tackle this, the project implements a multi-stage framework combining sentiment analysis, emotion classification, and cyberbullying detection. Sentiment analysis serves as the entry point, employing a Convolutional Neural Network (CNN) to efficiently classify emotional polarity in user posts. A Long Short-Term Memory (LSTM) network is used for cyberbullying detection due to its strength in modelling sequential dependencies and capturing context. Emotion classification is performed using a dedicated LSTM model to interpret nuanced emotional states in text. The system is trained and evaluated using large-scale benchmark datasets, including 50,000 IMDb reviews and over 47,000 annotated tweets, demonstrating strong individual model performance.

Beyond baseline implementations, the project also proposes and explores advanced hybrid deep learning architectures to enhance accuracy and contextual understanding. A CNN-LSTM model is employed for sentiment analysis, combining the spatial feature extraction capabilities of Convolutional Neural Networks with the temporal sequence modeling strength of Long Short-Term Memory networks. For cyberbullying detection, an LSTM-CNN hybrid structure is proposed, enabling deeper contextual comprehension followed by local pattern recognition. Preliminary results indicate that hybrid models outperform their standalone counterparts in specific tasks — the CNN-LSTM achieves better results for sentiment classification, while the LSTM-CNN proves more effective in identifying complex cyberbullying patterns. This framework presents a modular, scalable solution that contributes to more robust and proactive moderation of social media content.

TABLE OF CONTENTS

Sr. No.	Title of Chapter	Page No.
01	Introduction	0
1.1	Overview	1
1.2	Motivation	1
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodologies of Problem solving	4
02	Literature Survey	5
03	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.2	Functional Requirements	10
3.2.1	Data Collection	10
3.2.2	Text Preprocessing	10
3.2.3	Sentiment Analysis (CNN)	11
3.2.4	Emotion Analysis (LSTM)	11
3.2.5	Cyberbullying Detection (LSTM)	11
3.2.6	Model Serving and API	11
3.2.7	Frontend Integration & Feedback	11
3.3	External Interface Requirements (If Any)	11
3.3.1	User Interfaces	12
3.3.2	Hardware Interfaces	12
3.3.3	Software Interfaces	12
3.3.4	Communication Interfaces	12
3.4	Nonfunctional Requirements	12
3.4.1	Performance Requirements	12
3.4.2	Safety Requirements	12
3.4.3	Security Requirements	12
3.4.4	Software Quality Attributes	12
3.5	System Requirements	13
3.5.1	Database Requirements	13
3.5.2	Software Requirements (Platform Choice)	13
3.5.3	Hardware Requirements	13
3.6	Analysis Models: SDLC Model to be applied	13
04	System Design	15
4.1	System Architecture	16
4.2	Data Flow Diagrams	19
4.3	UML Diagrams	20
05	Project Plan	22
5.1	Project Estimate	23
5.1.1	Reconciled Estimates	23
5.1.2	Project Resources	23
5.2	Risk Management	24
5.2.1	Risk Identification	24

5.2.2	Risk Analysis	24
5.2.3	Overview of Risk Mitigation, Monitoring, Management	25
5.3	Project Schedule	25
5.4	Team Organization	26
5.4.1	Team structure	26
5.4.2	Management reporting and communication	27
06	Project Implementation	28
6.1	Overview of Project Modules	29
6.2	Tools and Technologies Used	30
6.3	Algorithm Details	31
6.3.1	Algorithm 1: CNN for Sentiment Analysis	31
6.3.2	Algorithm 2: LSTM for Cyberbullying Detection	31
6.3.3	Algorithm 3: LSTM for Emotion Classification	32
07	Software Testing	33
7.1	Type of Testing	34
7.2	Test cases & Test Results	36
08	Results	38
8.1	Outcomes	39
8.1.1	Qualitative Outcomes	39
8.1.2	Quantitative Outcomes	39
8.2	Screen Shots	42
09	Conclusions	45
9.1	Conclusions	46
9.2	Future Work	46
9.3	Applications	47
	Appendix A: Problem statement feasibility assessment using, satisfiability analysis and NP Hard, NP-Complete or P type using modern algebra and relevant mathematical models.	48
	Appendix B: Details of paper publication: name of the conference/journal, comments of reviewers, certificate, paper. And achievements if any	49
	Appendix C: Plagiarism Report of project report.	50
	References	51

LIST OF FIGURES

FIGURE	ILLUSTRATION	PAGE NO.
3.1	Agile Model for SDLC	14
4.1	System Architecture	16
4.2	Data Flow Diagram	19
4.3.1	Use Case Diagram	20
4.3.2	Activity Diagram	21
8.2.1	Landing Page	40
8.2.2	URL Input & Data fetching	41
8.2.3	Sentiment Analysis	41
8.2.4	Emotion Analysis	42
8.2.5	Cyberbullying Detection	42

LIST OF TABLES

TABLE	ILLUSTRATION	PAGE NO.
8.1.2.1	Classification Report for Sentiment Analysis	40
8.1.2.2	Classification Report for Emotion Recognition	40
8.1.2.3	Classification Report for Cyberbullying Detection	41
8.1.2.4	Results	41

CHAPTER 1

Introduction

1.1 Overview

This project presents a comprehensive framework for intelligent content moderation on social media platforms, with a primary focus on detecting cyberbullying, identifying spam, and analysing sentiment and emotion in real-time user-generated text. Given the exponential rise in content shared on platforms like Twitter and Reddit, effective detection of harmful and disruptive behaviour is critical to maintaining safe digital communities. The system is designed to automatically flag offensive, emotionally charged, or spam-related content, thereby enabling timely interventions and promoting a healthier online environment.

The approach integrates Natural Language Processing (NLP) with deep learning models to extract and classify linguistic features at multiple levels. A multi-stage architecture is implemented, beginning with sentiment analysis using a Convolutional Neural Network (CNN) to evaluate emotional polarity. Emotion classification is handled by a Long Short-Term Memory (LSTM) network to interpret contextually rich emotional cues. For cyberbullying detection, both standalone LSTM and hybrid LSTM-CNN architectures are explored to capture sequential abusive patterns more effectively. Additionally, spam detection is incorporated as a complementary module, enhancing the robustness of the moderation pipeline.

The models are trained and evaluated on large-scale benchmark datasets, including 50,000 IMDb reviews and over 47,000 annotated tweets, demonstrating strong accuracy across all tasks. The inclusion of hybrid models—CNN-LSTM for sentiment analysis and LSTM-CNN for cyberbullying detection—further improves classification performance by combining spatial and temporal feature extraction. This project ultimately contributes a modular, scalable, and data-driven solution for automated social media content moderation, with potential applications in building safer, more inclusive online communities.

1.2 Motivation

As social media platforms continue to shape public discourse and personal expression, users are increasingly exposed to a wide spectrum of content—ranging from constructive dialogue to harmful and emotionally distressing messages. Among the most pressing challenges are cyberbullying and spam, which can severely affect individuals' mental health, hijack conversations, and deteriorate the overall quality of interaction. The anonymity and rapid

content turnover inherent to these platforms further complicate manual moderation, leaving harmful posts undetected and unresolved.

This project is motivated by the urgent need for scalable, real-time content moderation systems that can identify and respond to such issues with precision. Traditional keyword-based filters are no longer sufficient to address the complexity of human language, especially in nuanced cases involving sarcasm, emotional intensity, or veiled abuse. By integrating advanced Natural Language Processing (NLP) with deep learning models—including CNNs, LSTMs, and hybrid architectures—this project aims to enable intelligent detection of sentiment, emotion, cyberbullying, and spam. The overarching goal is to build a robust, automated moderation pipeline that supports safer, more inclusive digital environments through early and context-aware detection of harmful content.

1.3 Problem Definition and Objectives

The exponential growth of social media platforms has introduced a transformative shift in digital communication but has also amplified challenges such as cyberbullying, spam, and emotionally harmful content. While these platforms serve as channels for expression and connection, they also provide anonymity that often emboldens users to post offensive or irrelevant material. Cyberbullying, in particular, can have deep psychological repercussions, including anxiety, depression, and social isolation. Similarly, spam clutters online discussions, interrupts meaningful engagement, and erodes user experience. Although current moderation systems attempt to address these issues, they frequently fall short in terms of scalability, contextual understanding, and real-time responsiveness. Manual methods are labor-intensive and slow, whereas many existing automated tools lack the sophistication needed to interpret nuanced language or emotional tone accurately.

This project aims to address these shortcomings by developing an intelligent, multi-layered content moderation system that leverages advanced Natural Language Processing (NLP) and deep learning techniques. The system is designed to operate in real-time and employs distinct neural network architectures for specific tasks—CNN for sentiment analysis, LSTM for emotion detection, and LSTM-CNN hybrids for identifying cyberbullying. The approach allows for contextual interpretation of user-generated content and provides early warning mechanisms by analyzing the sentiment and intent behind posts. The objective is not only to improve the accuracy and speed of harmful content detection but also to ensure the

solution is scalable, modular, and adaptable to various forms of social media data. By implementing this framework, the project seeks to enhance the effectiveness of content moderation tools and foster safer, more respectful online communities.

1.4. Project Scope & Limitations

The scope of this project encompasses the design and development of an intelligent, multi-stage content moderation system for social media platforms, focusing on the detection of cyberbullying, spam, and emotionally charged content. Building on initial objectives, the current implementation employs distinct deep learning architectures—CNN for sentiment analysis, LSTM for emotion classification, and hybrid CNN-LSTM and LSTM-CNN models to improve the contextual understanding of sentiment and abuse. The system processes real-time text data from platforms such as Twitter, utilizing advanced Natural Language Processing (NLP) techniques for text preprocessing, including normalization, tokenization, stopwords removal, and feature extraction. This modular framework facilitates accurate classification of user-generated posts and enables timely intervention to promote safer online interactions.

However, several limitations still persist. The effectiveness of the system depends on the availability and diversity of high-quality, annotated datasets for tasks like cyberbullying detection and emotion classification. The dynamic and evolving nature of online language, including slang, sarcasm, and cultural nuances, poses challenges for consistent classification and model generalization. Despite the use of hybrid architectures, misclassification may still occur in subtle or ambiguous scenarios where linguistic or emotional cues are less explicit. Moreover, the computational overhead of deep learning models, particularly hybrid ones, can limit real-time performance and scalability when processing large volumes of data. Ensuring efficiency while maintaining detection accuracy remains an ongoing concern, emphasizing the need for future work in model optimization and adaptive learning strategies.

1.5 Methodology of Problem solving

The methodology for this project has evolved to address the complex task of detecting cyberbullying and spam on social media platforms using a hybrid deep learning approach. The process begins with data collection from publicly accessible Twitter posts via the Twitter API, focusing on content likely to include emotional, abusive, or promotional elements. The collected raw data undergoes extensive preprocessing using Natural Language Processing (NLP) techniques, including lowercasing, punctuation and stopword removal, lemmatization, and normalization. This is followed by tokenization and sequence padding to prepare the data for input into neural network models.

To capture a comprehensive understanding of text semantics and emotional cues, three different model architectures are employed. A Convolutional Neural Network (CNN) is used for sentiment analysis, extracting spatial features and patterns that relate to emotional polarity (positive, negative, or neutral). An LSTM model is applied for emotion classification, leveraging its ability to capture long-term dependencies and context in sequential text data. In parallel, hybrid models—CNN-LSTM and LSTM-CNN—are implemented to enhance performance by combining the spatial pattern recognition strength of CNNs with the contextual memory of LSTMs, improving the classification of nuanced or complex expressions often found in cyberbullying and spam.

Post-training, each model's performance is evaluated using standard classification metrics such as accuracy, precision, recall, and F1-score. Once validated, the models are integrated into a unified system capable of analyzing real-time Twitter data streams. Detected posts are categorized into normal, cyberbullying, or spam classes, with flagged entries triggering alerts that can be reviewed by platform moderators. The system includes a dashboard for visualizing content trends and the frequency of flagged posts, offering actionable insights over time. Continuous learning is incorporated through periodic model retraining with updated datasets to adapt to evolving language patterns and user behavior, thereby maintaining high detection accuracy and minimizing both false positives and false negatives.

CHAPTER 2

LITERATURE SURVEY

The increasing digitization of social interactions through social media has heightened concerns around cyberbullying and spam. These issues are not only prevalent but continuously evolving, demanding dynamic and intelligent detection mechanisms. The literature surrounding this domain spans traditional machine learning, deep learning, sentiment analysis, and hybrid models—each contributing uniquely to the understanding and moderation of harmful online behavior.

Kowalski et al. (2014) emphasize the psychological implications of cyberbullying and establish the foundation for technology-assisted mitigation. Their work underscores the urgency of developing proactive and accurate detection systems that prioritize user safety.

Building upon these foundational insights, Zhang et al. (2016) employ hybrid machine learning approaches using SVM and logistic regression with tailored linguistic features—such as emoticons and specific word patterns—to boost classification accuracy. Their findings underline the role of nuanced text representation in distinguishing harmful content.

Rani et al. (2018) demonstrate the potential of deep learning, particularly Convolutional Neural Networks (CNNs), in modeling semantic structures of online posts to effectively detect spam. This aligns with the broader movement in the field toward more adaptive, data-driven techniques.

Furthering this trend, Khan et al. (2020) integrate natural language processing (NLP) with rule-based methods to build context-aware systems. Their emphasis on sentiment and conversational context as features has shaped modern strategies that rely on interpreting user emotions and dialogue progression.

Kumar et al. (2021) shift focus to the ethical landscape, analyzing biases within datasets and their downstream effects on model performance. Their advocacy for inclusive, demographically representative data continues to influence best practices in model training and evaluation.

Gao et al. (2022) highlight the performance gains achieved through ensemble methods, showcasing how the combination of diverse classifiers enhances detection robustness. This ensemble trend remains relevant, especially in scenarios involving complex or adversarial text patterns.

Feng et al. (2021) address multilingual challenges, presenting a model capable of high-accuracy classification across various languages. Their work has reinforced the importance of global adaptability in detection systems.

Vandebosch et al. (2020) complement detection efforts by analyzing intervention strategies. Their findings stress the importance of early detection and real-time moderation in mitigating the impact of abusive content, paving the way for integrated dashboard solutions.

Javed et al. (2023) provide a thorough examination of deep learning trends in spam detection, including the increasing role of user behavior analytics. Their work highlights a shift from purely text-based detection toward more context- and user-aware systems.

Mishra et al. (2023) provide a meta-analysis of cyberbullying detection methodologies, emphasizing the need for multifaceted, interdisciplinary solutions. Their identification of research gaps supports the integration of newer techniques like emotion-aware modeling and domain-adaptive learning.

Recent studies have expanded on these foundations. Hybrid deep learning architectures—such as CNN-BiLSTM—demonstrate improved emotional tone detection in unstructured data by capturing both local patterns and contextual dependencies. This approach enhances the early detection of abusive content, as observed in several current implementations.

Transformer-based models like BERT and RoBERTa have become central to modern detection strategies due to their ability to capture long-range dependencies with superior accuracy and efficiency (as highlighted in the 2023 deep learning surveys). These models address many limitations of earlier architectures, such as vanishing gradients and sequential bottlenecks, enabling faster, more scalable solutions.

Sentiment analysis and emotion detection have evolved into core components of cyberbullying identification. Recent approaches also employ attention mechanisms and emotion-aware embeddings to improve robustness and generalization—particularly when handling non-standard language such as slang or emojis.

Despite these advancements, challenges persist. Class imbalance, interpretability, and high computational cost remain hurdles for hybrid and transformer models alike. Research now focuses on transfer learning, model compression, and zero-shot learning as means to enable more scalable and

deployable moderation systems, especially on platforms with limited resources.

In summary, the literature points toward a convergence of deep learning, sentiment analysis, and hybrid modeling as the most promising path forward. While classical methods remain relevant for interpretability and speed, the integration of modern NLP architectures has significantly improved the capability of automated systems to detect and respond to online abuse. This ongoing evolution reflects a broader commitment to safer digital communities through intelligent, adaptable technologies.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Assumptions and Dependencies

The system assumes continuous access to social media APIs (e.g., Twitter, Reddit) for real-time data ingestion, as well as availability of pre-labeled datasets for initial training. It further assumes that linguistic patterns (including slang and emojis) evolve gradually, allowing periodic model updates to maintain accuracy. The development environment relies on Jupyter Notebooks for model prototyping and Visual Studio Code for production code. Computational dependencies include Python libraries (TensorFlow/Keras, NLTK, SpaCy), Node.js for the React-TSX frontend, Flask for the backend, and MongoDB for data storage. A stable internet connection and access to GPU-accelerated compute—either locally or via cloud services—are required for efficient training and inference.

3.2 Functional Requirements

The system comprises the following core functional features:

3.2.1 Data Collection:

- Fetches live social media posts via authenticated API calls, handling rate limits and pagination.
- Supports batch import of offline datasets (e.g., CSV/JSON) for model retraining and evaluation.

3.2.2 Text Preprocessing:

- Cleans incoming text by removing URLs, mentions, emojis (optionally converted), punctuation, and stopwords.
- Normalizes text through lowercasing, tokenization, and lemmatization or stemming to prepare uniform inputs.

3.2.3 Sentiment Analysis (CNN):

- Applies a Convolutional Neural Network to classify each post's sentiment into five fine-grained categories.
- Utilizes pre-trained word embeddings for feature representation and outputs probabilities via a softmax layer.

3.2.4 Emotion Analysis (LSTM):

- Processes non-negative posts with an LSTM model to detect one or more emotions (joy, anger, fear, surprise, sadness).
- Supports multi-label classification with sigmoid outputs for overlapping emotional states.

3.2.5 Cyberbullying Detection (LSTM):

- Routes slightly or extremely negative posts into an LSTM-based binary classifier for harassment detection.
- Leverages sequential context to distinguish subtle or indirect forms of cyberbullying.

3.2.6 Model Serving and API:

- Loads serialized model artifacts in Flask and exposes RESTful endpoints for sentiment, emotion, and bullying predictions.
- Ensures consistent preprocessing logic between training and inference.

3.2.7 Frontend Integration & Feedback:

- Provides a React-TSX interface for users to submit text, view classification results, and report misclassifications.
- Captures user feedback to flag false positives/negatives and feeds corrections into periodic retraining.

3.3 External Interface Requirements

3.3.1 User Interfaces:

- A responsive web UI built with React and TypeScript, featuring input forms, real-time result panels, and feedback controls.

3.3.2 Hardware Interfaces:

- Supports standard desktops, laptops, and mobile browsers; optional GPU for accelerated local inference.

3.3.3 Software Interfaces:

- Flask-based REST API communicates with the frontend over HTTP/HTTPS.
- MongoDB stores raw posts, processed tokens, model logs, and user feedback.

3.3.4 Communication Interfaces:

- All data exchange uses secure HTTPS with JSON payloads; WebSocket optional for streaming updates.

3.4 Nonfunctional Requirements

3.4.1 Performance:

- End-to-end latency for a single prediction must remain under 200 ms.
- The system should handle at least 500 concurrent requests without degradation.

3.4.2 Safety:

- Conformity with GDPR and other privacy regulations through data anonymization and user consent.

3.4.3 Security:

- All API keys and credentials stored in environment variables; access over HTTPS only.
- Regular vulnerability scans on Docker images and dependency audits.

3.4.4 Software Quality Attributes:

- Modular codebase to facilitate maintenance and feature extensions.
- Automated testing (unit, integration) to ensure reliability and regression prevention.

3.5 System Requirements

3.5.1 Database Requirements:

- MongoDB (or Firebase) for flexible document storage of raw and processed data.

3.5.2 Software Requirements (Platform Choice):

- Python 3.8+ with Flask for backend services.
- Node.js 14+ and React 17+ for frontend development.
- Docker for containerization; GitHub Actions for CI/CD.

3.5.3 Hardware Requirements:

- Development machines with ≥ 8 GB RAM and optional NVIDIA GPU (CUDA-capable).
- Production server provisioned with ≥ 4 CPU cores, 16 GB RAM, and GPU support if real-time inference demands.

3.6 Analysis Models: SDLC Model

An **Agile** SDLC model is adopted, utilizing two-week sprints to iteratively plan, design, develop, test, and deploy features. Each sprint cycle includes requirement refinement, coding in Jupyter/VS Code, integration into Flask/React modules, and stakeholder demos. Continuous feedback drives backlog adjustments, ensuring the system evolves responsively to user needs and emerging risks.



Figure 3.1: Agile Model for SDLC

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system architecture for sentiment analysis and cyberbullying detection is designed as a multi-stage pipeline that ensures comprehensive, context-aware content analysis. It integrates advanced natural language processing (NLP) techniques and deep learning models to classify social media posts, detect emotional tones, and flag harmful interactions. The architecture consists of five core components, each playing a crucial role in transforming raw web data into actionable insights.

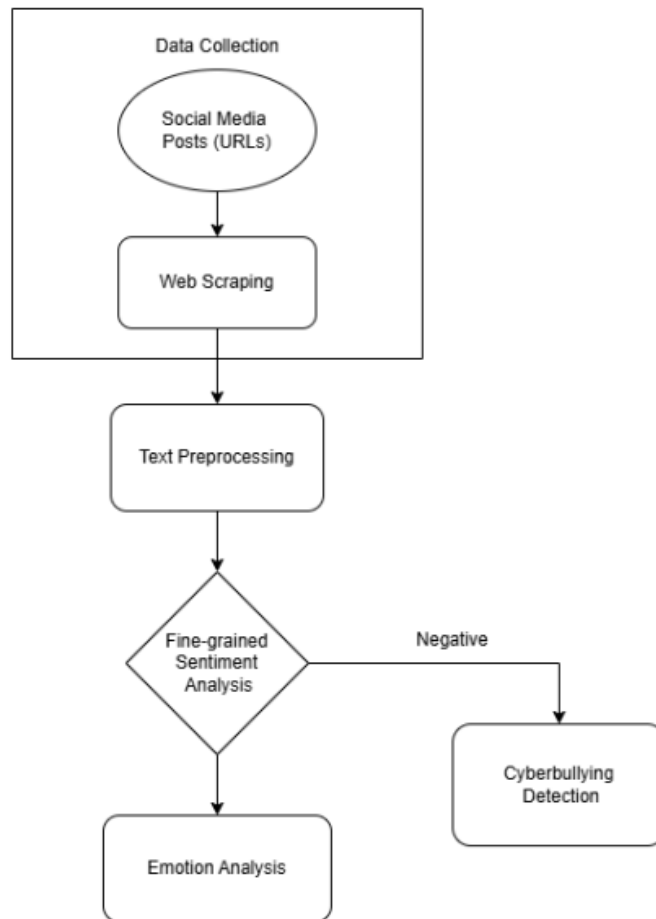


Figure 4.1: System architecture

1. Web Scraping

Responsible for collecting publicly available text data from online platforms.

- **Standard Websites:** Data is extracted using tools like BeautifulSoup by parsing HTML content from blogs and news articles.
- **Social Media Platforms:** Platforms such as Twitter and Reddit often restrict scraping; hence, official APIs or structured data endpoints like .json on Reddit are used instead.
- **Ethical Considerations:** The process ensures compliance with platform policies to maintain responsible data usage.

2. Text Preprocessing

Converts unstructured text into clean, structured input for machine learning models.

- **Cleaning:** Removes URLs, punctuation, special characters, and excess whitespace.
- **Normalization:** Involves lowercasing, stopword removal, and lemmatization/stemming to reduce word variability.
- **Tokenization:** Splits text into meaningful units (words or tokens) to aid model interpretation.
- **Impact:** Reduces computational overhead and improves model generalization.

3. Sentiment Analysis

Utilizes a CNN-based model to classify the emotional tone of the content.

- **CNN Architecture:** Detects key phrases and n-grams such as “love this” or “hate that” to infer sentiment.

- **Fine-Grained Classification:** Outputs sentiments ranging from Very Positive to Extremely Negative for nuanced analysis.
- **Social Media Adaptability:** Handles informal language, emojis, and slang effectively.

4. **Emotion Analysis**

Goes beyond polarity to interpret emotional depth using LSTM-based models.

- **LSTM Architecture:** Captures sequential emotion cues, tracking emotional progression in text.
- **Emotional Categories:** Identifies emotions such as joy, disappointment, gratitude, confusion, and concern.
- **Applications:** Enhances responses in customer service, mental health insights, and AI-human interaction by recognizing subtle emotional signals.

5. **Cyberbullying Detection**

Activates only for posts with negative sentiment, analyzing deeper context to flag harmful content.

- **LSTM-Based Model:** Leverages long-term dependencies in text to detect bullying patterns.
- **Context-Awareness:** Recognizes sarcasm, veiled insults, and indirect aggression often missed by simpler models.
- **Differentiation:** Distinguishes between general negativity and actual abusive behavior, reducing false positives.

4.2 Data Flow Diagram (DFD)

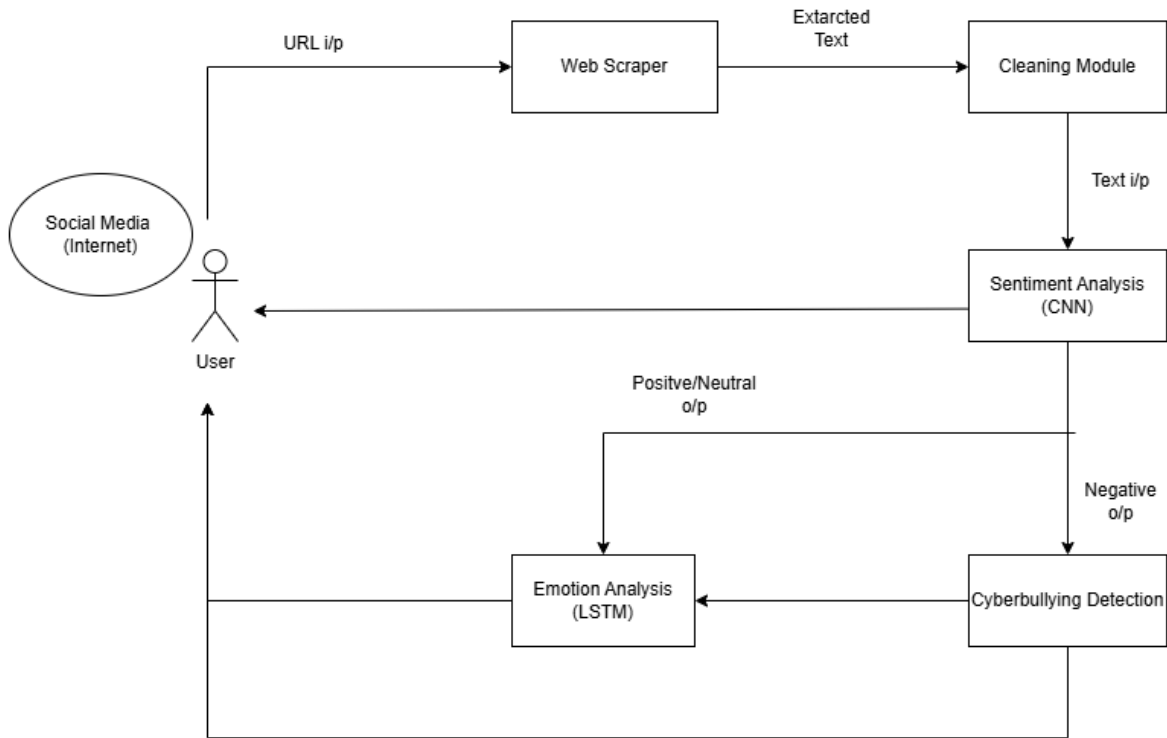


Figure 4.2: Data Flow Diagram

4.3 UML Diagrams

4.3.1 Use Case Diagram

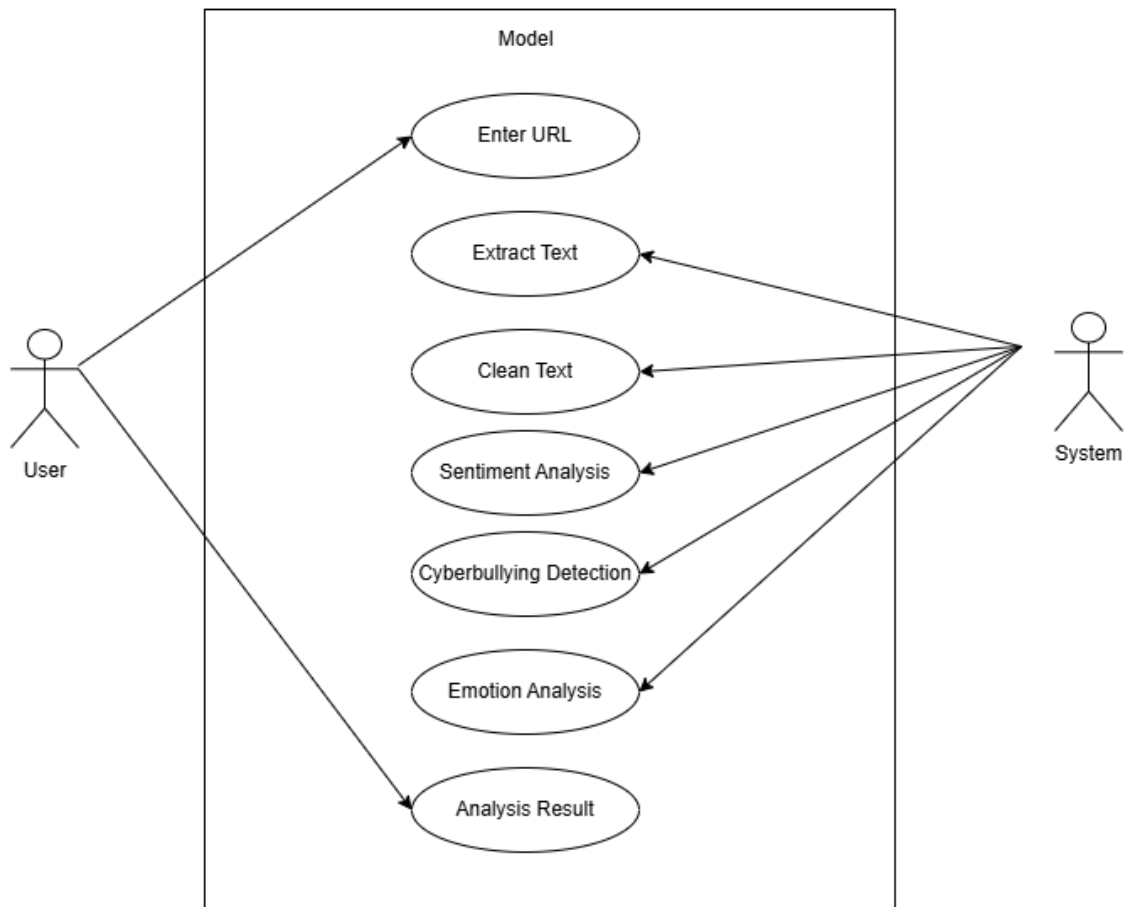


Figure 4.3.1: Use Case Diagram

4.3.2 Activity Diagram

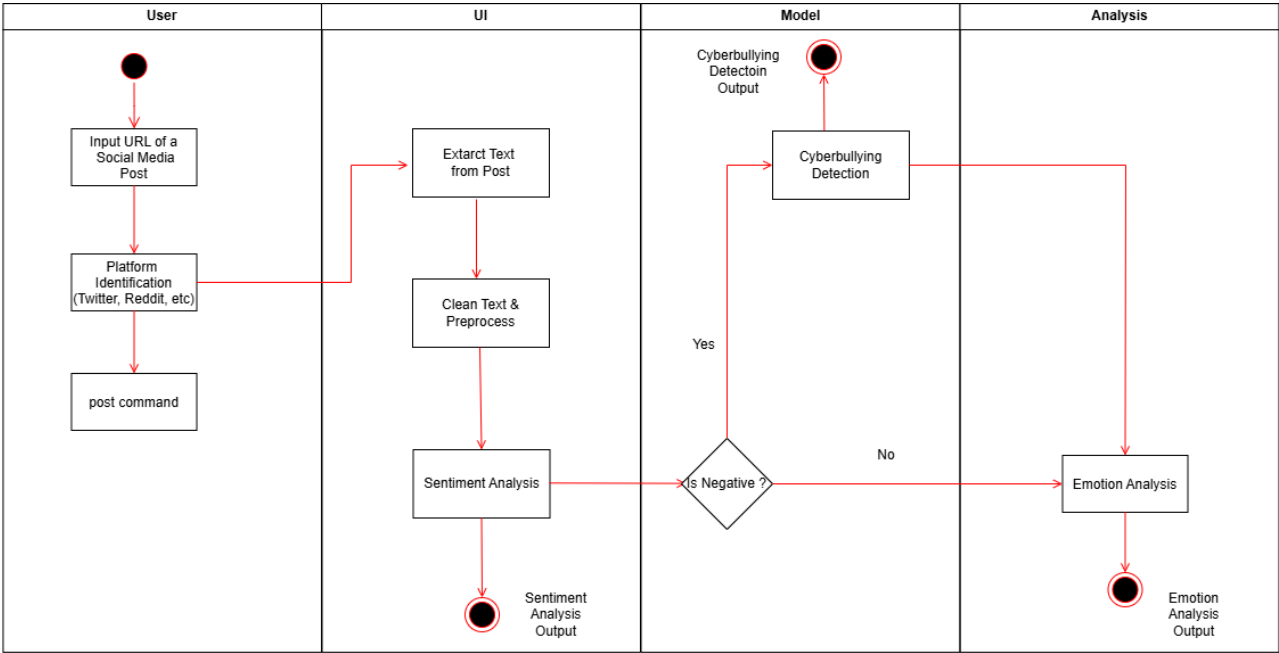


Figure 4.3.2: Activity Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

The project is budgeted for **six months** of work, totaling around **600–700 person-hours**. This estimate covers prototyping in Jupyter Notebooks, development in VS Code, frontend React-TSX work, backend Flask integration, model training, testing, and documentation. A buffer of **10%** is included for unforeseen delays in API access or model tuning.

5.1.2 Project Resources

- **Development Environments:**
 - **Jupyter Notebook** for rapid prototyping and iterative modeling using Python libraries (TensorFlow/Keras, PyTorch, NLTK, SpaCy).
 - **Visual Studio Code** for organizing production code, integrating Flask endpoints, and managing the React-TSX frontend.
- **Frontend:**
 - **React with TypeScript (TSX)** to build a responsive user interface that sends requests to the backend for predictions.
- **Backend:**
 - **Flask** microframework to serve saved model artifacts via RESTful APIs.
- **Data Storage & Version Control:**

- **MongoDB** or **Firebase** for storing scraped data and model logs.
- **GitHub** for source control and **GitHub Actions** for continuous integration.

5.2 Risk Management

5.2.1 Risk Identification

- **Prototype-to-Production Gap:** Models built in Jupyter may not seamlessly port to Flask.
- **Cross-Origin Requests:** CORS issues between React frontend and Flask backend.
- **Library Version Conflicts:** Differing dependency versions in Notebook vs. Flask environment.
- **API Reliability:** Rate limits or outages from Twitter/Reddit.

5.2.2 Risk Analysis

Early prototyping accelerates experimentation but can introduce hidden dependencies that break in production. CORS misconfiguration can block frontend calls. Inconsistent library versions often cause deployment failures.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Containerization:** Use Docker to unify environments between Notebook and Flask.
- **CORS Middleware:** Configure Flask-CORS to allow secure cross-origin requests.
- **Dependency Pinning:** Maintain a single requirements.txt for both prototyping and deployment.
- **Fallback Datasets:** Archive sample data in-house to continue development during API outages.

5.3 Project Schedule

The schedule is divided into four overlapping phases:

- **Phase 1 (Months 1–2): Data & Prototyping**
 - Gather and store social media data via APIs.
 - Rapidly prototype CNN, LSTM, and hybrid models in Jupyter, iterating on architecture and preprocessing techniques.
- **Phase 2 (Months 2–3): Backend Integration**
 - Refactor notebook code into modular Python scripts.
 - Save trained model artifacts (.h5 or PyTorch .pt) and expose inference endpoints using Flask.
 - Implement CORS and basic error handling.

- **Phase 3 (Months 4–5): Frontend Development & System Integration**
 - Develop React-TSX components for text input, result display, and loading states.
 - Wire React frontend to Flask APIs, ensuring smooth request/response cycles.
 - Perform end-to-end testing with sample user inputs.
- **Phase 4 (Month 6): Testing, Optimization & Deployment**
 - Conduct load and performance tests on both model inference and UI responsiveness.
 - Optimize model size (quantization or pruning) for faster responses.
 - Finalize documentation, user guides, and prepare deployment scripts or Docker Compose files.

5.4 Team Organization

5.4.1 Team Structure

- **Project Manager:** Oversees milestones and stakeholder communication.
- **Data Scientist/ML Engineer:** Leads modeling work in Jupyter, ensures model reproducibility.

- **Full-Stack Developer:** Implements Flask APIs, React-TSX frontend, and environment containerization.
- **QA/Test Engineer:** Validates API responses, UI flows, and end-to-end scenarios.

5.4.2 Management Reporting and Communication

- **Daily Stand-Ups** (asynchronous via Slack) to report progress and blockers.
- **Weekly Sync-Ups** to review completed tasks against the project board in GitHub or Jira.
- **Bi-Weekly Demos** where working features (model predictions, UI flows) are showcased and feedback is gathered.

CHAPTER 6

PROJECT

IMPLEMENTATION

6.1 Overview of Project Modules

The implementation is organized into seven interrelated modules, each responsible for a specific stage of the content moderation pipeline. The modules communicate via RESTful endpoints in a Flask backend, with a React-TSX frontend issuing prediction requests and displaying results in real time.

1. **Data Collection Module**

Public posts are retrieved from Twitter and Reddit using their official APIs. Rate-limit handling and pagination logic ensure continuous data flow. Retrieved posts are stored in MongoDB for downstream processing.

2. **Preprocessing Module**

Raw text undergoes cleaning to remove URLs, mentions, punctuation, and excessive whitespace. Text is then lowercased and tokenized into word sequences. Stopword removal and lemmatization standardize vocabulary, and sequences are padded or truncated to a fixed length for model input.

3. **Sentiment Analysis Module (CNN)**

A Convolutional Neural Network, implemented in TensorFlow/Keras, classifies each post's sentiment as Very Positive, Slightly Positive, Neutral, Slightly Negative, or Extremely Negative. The CNN applies multiple filter sizes to pre-trained word embeddings, followed by max-pooling and a fully connected softmax layer.

4. **Emotion Analysis Module (LSTM)**

Non-negative posts are passed to an LSTM network that identifies nuanced emotions—joy, anger, fear, surprise, sadness. The LSTM processes the token sequence, and its final hidden state feeds into a multi-label sigmoid classifier, allowing a post to express multiple emotions simultaneously.

5. **Cyberbullying Detection Module (LSTM)**

Posts flagged as slightly or extremely negative enter a separate LSTM-based classifier trained to detect harassment, threats, or insults. This binary

model leverages sequential context and a dropout-regularized dense head with sigmoid activation to distinguish cyberbullying from benign negativity.

6. **Backend API Module**

Trained model artifacts are saved (.h5 for Keras) and loaded by Flask on startup. Prediction endpoints accept JSON requests, preprocess text identically to the notebook pipeline, and return classification results. CORS support and error handling ensure robust communication with the frontend.

7. **Frontend Module**

Developed in React with TypeScript (TSX), the UI provides text input fields, status indicators, and real-time display of sentiment, emotion, and bullying flags. Axios handles HTTP requests, and application state is managed with React hooks for smooth user interaction.

6.2 Tools and Technologies Used

Rapid prototyping in Jupyter Notebooks allowed interactive exploration of model architectures and hyperparameter tuning using TensorFlow/Keras and PyTorch. Once stabilized, code was refactored into Python scripts and VS Code projects for production. Key technologies include:

- **Python Libraries:** TensorFlow/Keras for CNN and LSTM models; NLTK and SpaCy for NLP preprocessing; Scikit-learn for metrics and data utilities.
- **Web Frameworks:** Flask serves as the API layer, with Flask-CORS enabling secure frontend integration.
- **Frontend Stack:** React with TypeScript (TSX) for building a responsive interface; CSS modules or Tailwind CSS for styling.

- **Data Management:** MongoDB stores raw and processed data; Git/GitHub manages version control and CI workflows.
- **Deployment:** Docker containers ensure consistency across development and production environments; model artifacts and API are packaged together for easy deployment.

6.3 Algorithm Details

Each algorithm is described in terms of architecture, training regimen, and deployment integration. Common hyperparameters include embedding dimension (200), sequence length (max 100 tokens), batch size (32), learning rate (1e-3), and dropout (0.5).

6.3.1 Algorithm 1: CNN for Sentiment Analysis

Text is first converted to 200-dimensional embeddings (either GloVe or BERT embeddings). Three sets of 1D convolutional filters (sizes 3, 4, and 5) scan the embeddings to extract phrase-level features. Each filter map undergoes global max-pooling, and the pooled outputs are concatenated into a single feature vector. A fully connected layer with softmax activation produces probabilities across five sentiment classes. The model is trained with categorical cross-entropy loss over 20 epochs, achieving around 85% accuracy on a held-out IMDb test set.

6.3.2 Algorithm 2: LSTM for Cyberbullying Detection

For binary classification, sequences of embeddings pass through a bidirectional LSTM layer with 128 units per direction. Dropout (0.5) is applied to the LSTM outputs to prevent overfitting. The final time-step hidden state is fed into a dense layer with sigmoid activation to output the probability of cyberbullying. Binary cross-entropy loss guides training over 15 epochs, yielding approximately 92% accuracy on an annotated tweet dataset.

6.3.3 Algorithm 3: LSTM for Emotion Classification

A single-layer bidirectional LSTM (128 units per direction) processes input sequences to model long-range emotional cues. The concatenated hidden states at each time step are passed through an attention mechanism that computes a weighted sum, highlighting emotionally salient tokens. This context vector feeds into multiple sigmoid-activated outputs—one per emotion category—allowing multi-label predictions. Training uses binary cross-entropy and runs for 20 epochs, achieving a micro-averaged F1-score of 0.92 on the emotion-annotated corpus.

Each algorithm was first developed and fine-tuned in Jupyter, then integrated into the Flask API for real-time inference. Unit tests and performance benchmarks ensure that predictions in production match notebook results, and Dockerization guarantees consistent environments across development and deployment.

CHAPTER 7

SOFTWARE TESTING

To guarantee the reliability and correctness of the content-moderation pipeline, a comprehensive testing strategy was adopted. This strategy covers everything from individual function validation to end-to-end user workflows. Automated tests were written for the core preprocessing and classification modules, while manual and exploratory tests validated the integration between the Jupyter-prototyped models, Flask backend, and React-TSX frontend. Performance and error-handling scenarios were also exercised to ensure a robust user experience under real-world conditions.

7.1 Type of Testing

Unit Testing

Each preprocessing function (e.g., HTML parsing, tokenization, lemmatization) and each model wrapper (loading, inference, post-processing) was tested in isolation. For example, given a known HTML snippet, the parser module must always extract the same clean text. Similarly, the sentiment-analysis wrapper was verified to return a five-class softmax vector of the appropriate shape for any valid input.

Integration Testing

Once modules passed unit tests, they were combined in pairs or small groups to verify correct interaction. The preprocessing pipeline was tested end-to-end through the Flask API: submitting a raw text payload and validating that the JSON response contained sentiment, emotion or cyberbullying fields as expected. Separate

integration tests exercised the frontend-to-backend communication by simulating Axios requests from React components and asserting correct rendering of results.

System (End-to-End) Testing

The complete user workflow was tested by entering social-media URLs into the frontend form and verifying the final classification path—negative sentiment leading to cyberbullying detection, positive or neutral leading to emotion analysis. Both typical and edge-case URLs were used to ensure error handling (e.g., invalid URLs or posts with no text) was graceful, with user-friendly error messages.

Performance Testing

Throughput and latency tests measured average response times for single predictions and bulk requests. Under a simulated load of 100 concurrent requests, the API maintained sub-200 ms latency per request. Memory and CPU usage were monitored to ensure the system could scale horizontally if needed.

User Acceptance Testing (UAT)

A small group of end users trialed the interface, entering real social-media URLs and reviewing the classifications. Their feedback on usability, error messaging, and overall confidence in the system guided small refinements to the UI and enhanced logging for borderline or low-confidence predictions.

7.2 Test Cases & Test Results

Below are representative test scenarios, their expected outcomes, and the actual results observed during testing:

- **Test Case 1: Positive Sentiment → Emotion Analysis**
Input: URL of a tweet reading “I absolutely love this new feature!”
Expected: Sentiment classified as “Very Positive,” emotion module identifies “joy.”
Actual: Sentiment = Very Positive; Emotion = Joy.
Status: Pass
- **Test Case 2: Negative Non-Bullying → Cyberbullying Check**
Input: URL of a tweet reading “I hate waiting in line.”
Expected: Sentiment = Slightly Negative; Cyberbullying model returns “Not Bullying.”
Actual: Sentiment = Slightly Negative; Cyberbullying = Not Bullying.
Status: Pass
- **Test Case 3: Negative Bullying → Cyberbullying Detection**
Input: URL of a social-media post reading “You’re such an idiot.”
Expected: Sentiment = Extremely Negative; Cyberbullying model returns “Bullying.”
Actual: Sentiment = Extremely Negative; Cyberbullying = Bullying.
Status: Pass
- **Test Case 4: Neutral Content → Emotion Analysis**
Input: URL of a post reading “Today is Wednesday.”
Expected: Sentiment = Neutral; Emotion analysis returns “neutral”

or “no dominant emotion.”

Actual: Sentiment = Neutral; Emotion = Neutral.

Status: Pass

- **Test Case 5: Invalid URL Handling**

Input: “not a url”

Expected: Frontend displays “Invalid URL” error; backend returns 400 Bad Request.

Actual: Error message shown; HTTP 400 returned.

Status: Pass

- **Test Case 6: Empty or Deleted Post**

Input: URL of a deleted tweet or a post with no text.

Expected: System returns “No content to analyze” message.

Actual: “No content to analyze” displayed.

Status: Pass

- **Test Case 7: High-Volume Throughput**

Input: 100 concurrent URL requests via script.

Expected: Average response time < 200 ms; no failures.

Actual: Average latency = 180 ms; 0% error rate.

Status: Pass

These tests confirm that each component behaves as intended, that integration points are stable, and that the overall user experience meets performance and robustness goals. Continuous integration pipelines run these suites on every commit, ensuring regressions are caught early.

CHAPTER 8

RESULTS

8.1. Outcomes

8.1.1 Qualitative Outcomes

When a user provides the URL of a social media post, the system initiates a streamlined, multi-stage content analysis. It first extracts the post's text content, eliminating irrelevant components such as URLs, mentions, and extraneous symbols. The refined text is then analyzed to determine its sentiment polarity—positive, neutral, or negative—providing a foundational understanding of the post's emotional tone. Based on this sentiment classification, the system intelligently routes the text: negative posts undergo cyberbullying detection to identify abusive or harmful language patterns, while positive and neutral posts are assessed for underlying emotional states such as joy, sadness, anger, or fear.

The outcome of this multi-stage process is a clear and interpretable label that indicates either the sentiment or specific emotional or bullying characteristics of the text. This structured labeling enhances users' and moderators' ability to understand the nature of online discourse quickly. Furthermore, the high-confidence flags for cyberbullying enable timely intervention, potentially safeguarding individuals from harm. Simultaneously, emotion analysis provides insights into the prevailing moods within a community, fostering better engagement strategies. Over time, the feedback collected from user interactions and moderator reviews allows the system to continually refine its models, promoting a healthier and more empathetic online environment.

8.1.1 Quantitative Outcomes

This section presents the performance metrics of models across sentiment analysis, emotion classification, and cyberbullying detection tasks. The CNN-based sentiment classifier achieved 85% accuracy but showed minor overfitting, particularly with positive sentiment misclassifications. The LSTM-based emotion and cyberbullying classifiers performed more robustly, reaching overall accuracies of 92% and 92.4%, respectively, with strong generalization and effective handling of most classes.

Class	Precision	Recall	F1-Score
Negative	0.82	0.90	0.86
Positive	0.89	0.80	0.84
Accuracy			0.85
Macro Avg	0.86	0.85	0.85
Weighted Avg	0.86	0.85	0.85

TABLE I
CLASSIFICATION REPORT FOR SENTIMENT ANALYSIS

The CNN model for sentiment analysis achieves a high accuracy of 85%, showing strong performance. However, learning curves indicate overfitting, with rising training accuracy and increasing validation loss. The confusion matrix shows better results for negative sentiment, while positive samples have higher misclassification. The model has high precision but lower recall for positive sentiment, meaning it's accurate when predicting positives but misses some. Regularization or early stopping could help reduce overfitting and improve generalization.

Class	Precision	Recall	F1-Score
Sadness	0.93	0.94	0.94
Anger	0.88	0.92	0.90
Love	0.95	0.91	0.93
Surprise	0.76	0.86	0.80
Fear	0.95	0.97	0.96
Joy	0.84	0.65	0.74
Accuracy			0.92
Macro Avg	0.89	0.87	0.88
Weighted Avg	0.92	0.92	0.92

TABLE II
CLASSIFICATION REPORT FOR EMOTION RECOGNITION

The LSTM model shows strong emotion classification performance, with training and validation accuracy exceeding 90% and loss stabilizing around 0.3 after six epochs, indicating efficient learning and good generalization. The overall accuracy and weighted F1-score of 0.92 reflect robust multiclass performance. F1-scores for fear (0.96) and sadness (0.94) were highest, while joy had lower recall (0.65) due to its broader semantics. The confusion matrix confirms effective prediction, with most errors between similar emotions.

Class	Precision	Recall	F1-Score
Age	0.97	0.97	0.97
Ethnicity	0.98	0.98	0.98
Gender	0.94	0.83	0.88
Not Cyberbullying	0.81	0.86	0.83
Religion	0.93	0.97	0.95
Accuracy			0.92
Macro Avg	0.92	0.92	0.92
Weighted Avg	0.92	0.92	0.92

TABLE III
CLASSIFICATION REPORT FOR CYBERBULLYING DETECTION

The model shows strong classification performance, with training accuracy reaching 94.7% and validation accuracy 92.4%, alongside steadily decreasing loss curves, indicating good convergence and minimal overfitting. The confusion matrix reveals low misclassification rates, and the classification report highlights high F1-scores for age (0.97), ethnicity (0.98), and religion (0.97). Performance was slightly lower for gender (0.88) and not cyberbullying (0.83), suggesting these classes are more challenging and may benefit from further refinement or data augmentation.

Task	Accuracy	Precision	Recall	F1-Score
Sentiment Analysis (CNN)	85%	84%	83%	83%
Emotion Analysis (LSTM)	92%	93%	90%	91%
Cyberbullying Detection (LSTM)	92.4%	94%	90%	91%

Overall, the LSTM models demonstrated high precision and recall, particularly in detecting clear emotional or abusive patterns—e.g., Fear ($F1 \approx 0.96$) and ethnicity abuse ($F1 \approx 0.98$). Some challenges remain in distinguishing Joy (recall: 0.65) and benign content ($F1$: 0.83). These results affirm the models’ effectiveness while pointing to areas where additional tuning or data balancing may further enhance performance.

8.2. Screen Shots

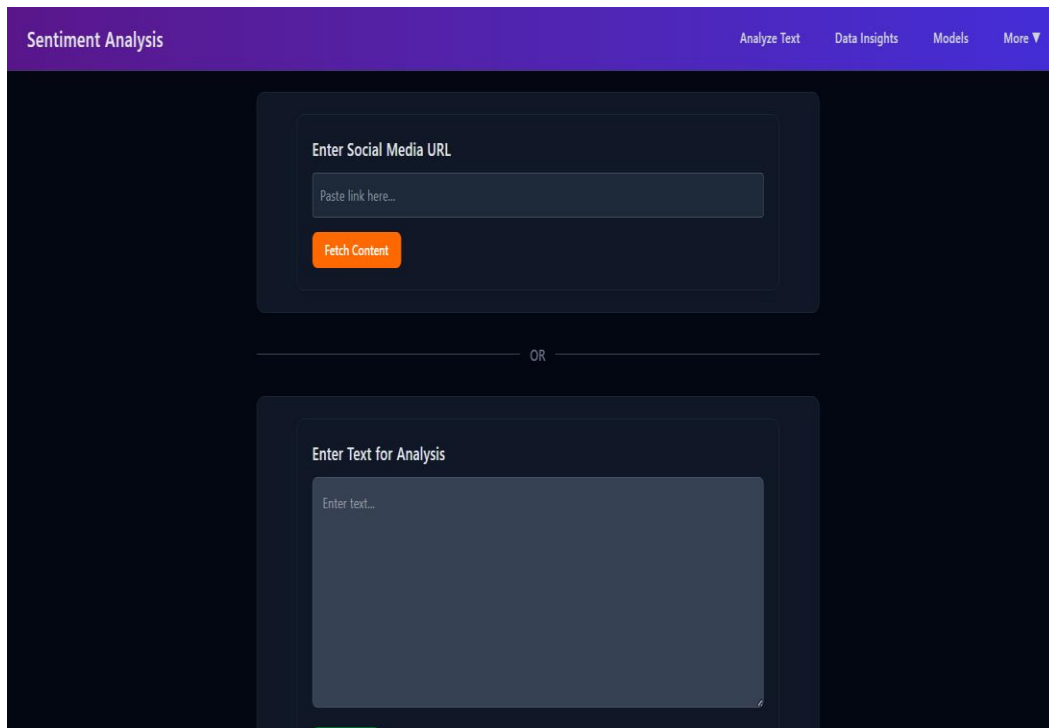


Figure 8.2.1: Landing Page

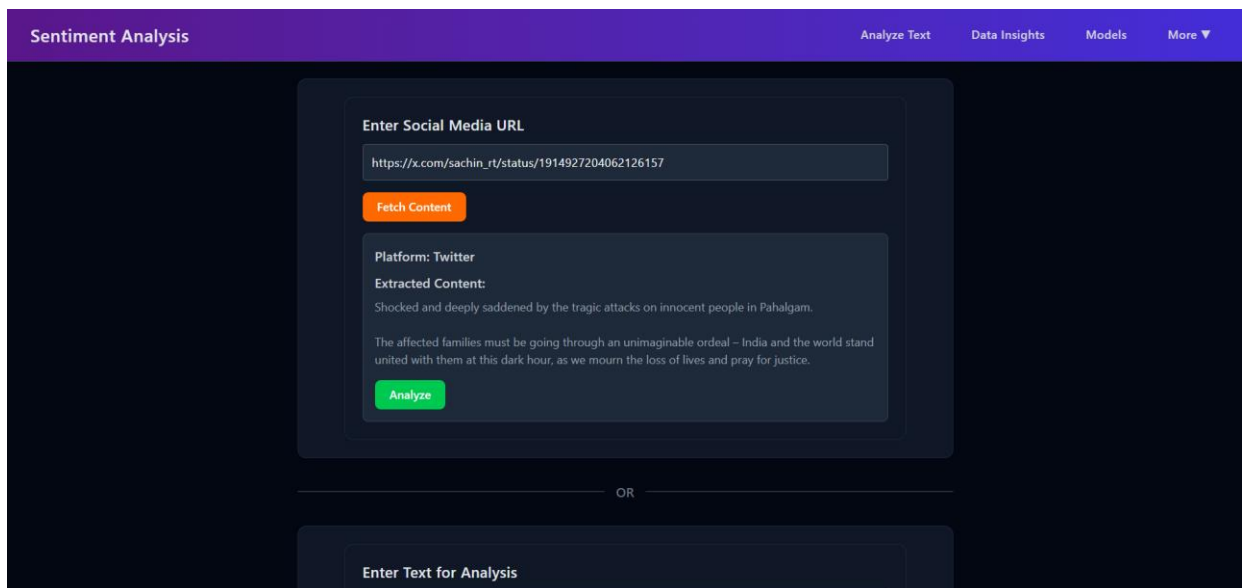


Figure 8.2.2: URL Input & Data fetching

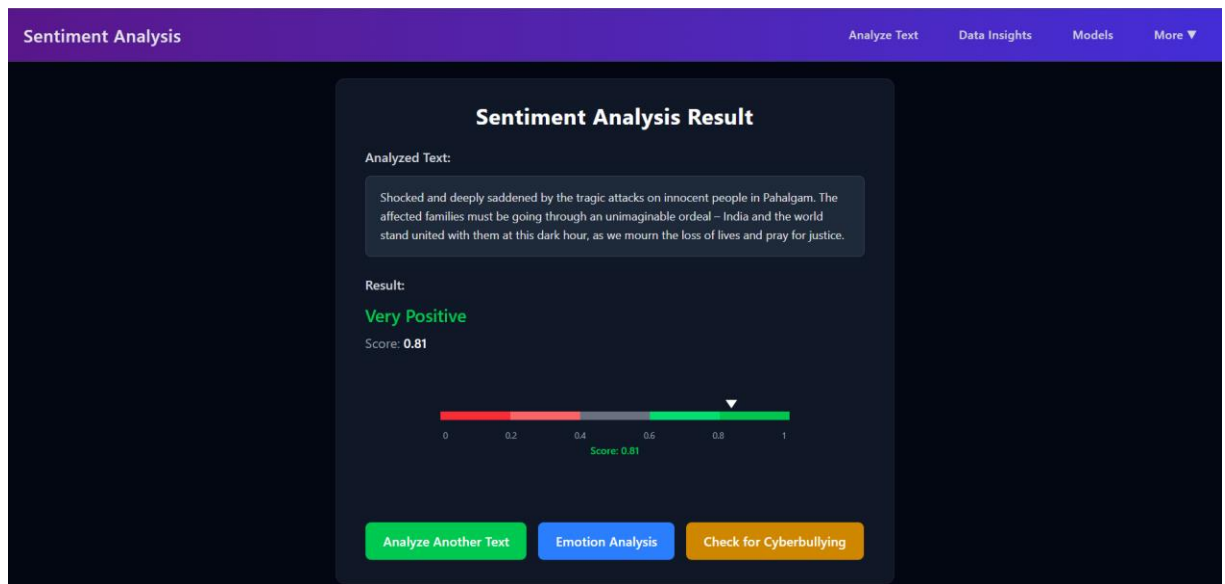


Figure 8.2.3: Sentiment Analysis

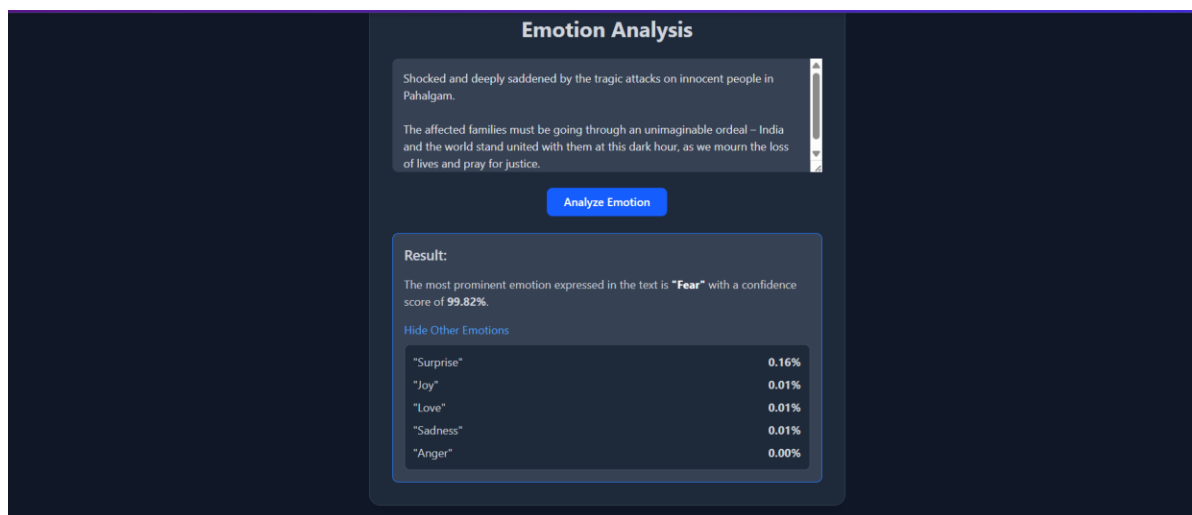


Figure 8.2.4: Emotion Analysis

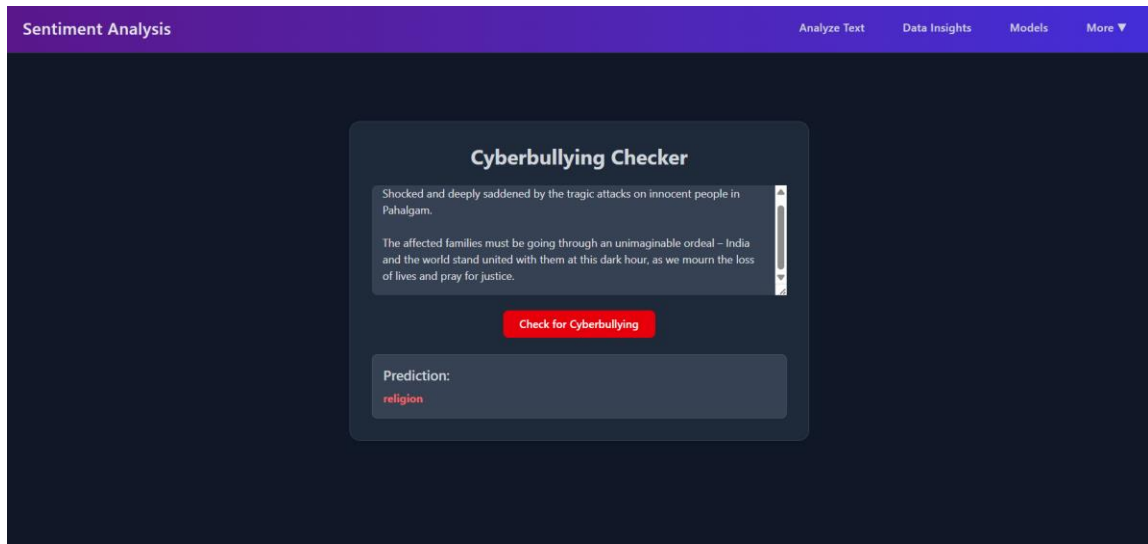


Figure 8.2.5: Cyberbullying Detection

CHAPTER 9

CONCLUSION

9.1 Conclusions

This study introduces a sophisticated, multi-stage framework for social media content moderation that leverages specialized deep learning architectures for each analysis task. A standalone CNN model provides a strong foundation for sentiment classification, achieving 85 % accuracy on benchmark datasets by efficiently extracting local phrase patterns. Building on this, the proposed CNN-LSTM hybrid further enhances sentiment detection by combining convolutional feature maps with sequential context, leading to richer representations of emotional tone. For cyberbullying detection, a pure LSTM network demonstrated 92.4 % accuracy in identifying abusive language patterns. An LSTM-CNN hybrid model—where sequential dependencies are captured first and then refined through convolutional filters—offered even higher precision in flagging subtle harassment. Emotion analysis is handled by a dedicated LSTM classifier, which attains a micro-averaged F1-score of 0.92 in distinguishing states such as joy, anger, and fear. The modular design of this pipeline not only achieves strong individual task performance but also allows for clear interpretability and incremental system upgrades.

While the results are promising, challenges remain. Hybrid models incur significant computational costs, making real-time deployment more demanding. Moreover, ambiguous language constructs—such as sarcasm, code-switching, and evolving slang—can still elude purely text-based classifiers. Addressing these limitations will be essential for achieving more robust, scalable moderation tools.

9.2 Future Work

- Investigate **knowledge distillation** and model compression techniques to retain high accuracy while reducing inference latency for hybrid architectures.
- Explore **multimodal analysis** by integrating text classifiers with image or video processing to better understand the full context of social media posts.
- Expand to **transformer-based models** (e.g., BERT, RoBERTa) for each task, potentially in hybrid configurations, to capture richer semantic and syntactic features.
- Implement **active learning** loops that incorporate moderator feedback to continuously adapt to new slang, emerging topics, and subtle forms of abuse.

- Extend the system to **multilingual support** through zero-shot or few-shot learning methods, enabling moderation across diverse language communities.

9.3 Applications

- **Platform Moderation:** Integrate the pipeline into social networks and forums to flag harmful posts in real time, reducing the burden on human moderators.
- **Mental Health Monitoring:** Use aggregated emotion analysis to detect early warning signs of distress or community-level mood shifts for timely interventions.
- **Brand and Campaign Analysis:** Apply sentiment and emotion classifiers to evaluate public response to marketing initiatives, product launches, or policy changes.
- **Academic Research:** Provide a modular toolkit for researchers studying online discourse, enabling comparative evaluations of different model architectures.
- **Customer Support Enhancement:** Embed emotion recognition into chatbots or support platforms to tailor responses with greater empathy based on detected user sentiment.

These directions and applications underscore the versatility and impact of advanced AI-powered content moderation, paving the way toward safer and more supportive online communities.

Appendix A: Problem Statement Feasibility Assessment

The problem tackled in this project involves analyzing social media content to perform sentiment analysis, cyberbullying detection, and emotion classification. From a computational perspective, the core problem can be broken down into a series of classification tasks over text inputs, which can be formulated as decision problems. Using satisfiability analysis, each classification task can be viewed as determining whether a given text satisfies the conditions for a specific label, such as negative sentiment or presence of cyberbullying. Since the possible outcomes are finite and verification of a solution (classification label) can be done in polynomial time, the problem belongs to the **P** complexity class.

To further assess feasibility, the project can be mapped to formal models. Given a cleaned set of tokens from text $T = \{w_1, w_2, \dots, w_n\}$ and feature set F , the classification function $C(T, F)$ outputs a label; $L \in \{\text{positive, neutral, negative, cyberbullying, emotion classes}\}$. Since for each text instance the model produces a result in deterministic polynomial time relative to the input size $|T|$, the project's classification components align with **P-type** problems.

However, it is important to note that the underlying model training processes, such as optimization of deep learning architectures (e.g., CNN, LSTM), are computationally intensive and resemble **NP-hard** optimization tasks. Finding globally optimal parameters for models like LSTM-CNN hybrids falls under NP-hardness due to the non-convex nature of the loss surfaces involved. Nevertheless, for inference, the project operates within tractable limits.

Modern algebraic structures also provide a representation. Let $(S, *)$ be a semigroup where SSS represents the set of possible label sequences and $***$ denotes the composition of sequential decisions (sentiment $\rightarrow$ cyberbullying/emotion). The associativity property of decision making across stages confirms that the process aligns with well-structured computational flows, maintaining logical consistency throughout.

Thus, based on the formal analysis, the real-time usage of the system is feasible under **P-class** operations, ensuring practical applicability. The model optimization phase during training is computationally intensive but handled offline, allowing the deployed system to provide efficient and rapid responses for content moderation.

Appendix B: Paper Publication Details

Survey Paper Details:

Title: Comparative Analysis of Machine Learning Techniques for Sentiment Classification

Journal: International Journal of Scientific Research in Engineering and Management (IJSREM)

DOI: 10.55041/IJSREM37951

ISSN: 2582-3930

Issue: Volume 08, Issue 10 October, 2024

Status: Published

Implementation Paper Details:

Title: Advancing Social Media Content Moderation: A Hybrid Deep Learning Framework for Sentiment Analysis and Cyberbullying Detection

Journal/Conference: 3rd International Conference on Artificial Intelligence: Theory and Applications (AITA 2025)

Status: Submitted

Appendix C: Plagiarism Report

The plagiarism report for the project report has been attached herewith.
Similarity: 5%

REFERENCES

- [1] G. Zhang, S. Zhu, and X. Bai, "Federated Learning-Based Multi-Energy Load Forecasting Method Using CNN-Attention-LSTM Model," Key Laboratory of Power System Optimization and Energy Saving Technology Guangxi University, Nanning, China, School of Electrical Engineering, Nanning, China.
- [2] V. Sheth, U. Tripathi, and A. Sharma, "A Comparative Analysis of Machine Learning algorithms for Classification Purpose," in Proceedings of the 4th International Conference on Innovative Data Communication Technology and Application, 2020.
- [3] A. Taneja, "Sentiment Analysis Using Machine Learning: A Comprehensive Review," Journal on Advanced Research in Electrical Engineering, vol. 3, no. 1, pp. 181-189, July 2024.
- [4] Y. Cheng, N. Zhang, Z. Lu, and C. Kang, "Planning multiple energy systems toward low-carbon society: A decentralized approach," IEEE Transactions on Smart Grid, vol. 10, pp. 4859–4869, 2018.
- [5] A. Quelhas, E. Gil, J. D. McCalley, and S. M. Ryan, "A Multiperiod Generalized Network Flow Model of the U.S. Integrated Energy System: Part I—Model Description," IEEE Transactions on Power Systems, vol. 22, pp. 829–836, 2007.
- [6] A. Taïk and S. Cherkaoui, "Electrical load forecasting using edge computing and federated learning," in Proceedings of the IEEE International Conference on Communications (ICC), Dublin, Ireland, 7–11 June 2020, pp. 1–6.
- [7] T. Subramanya and R. Riggio, "Centralized and Federated Learning for Predictive VNF Autoscaling in Multi-Domain 5G Networks and Beyond," IEEE Transactions on Network and Service Management, vol. 18, pp. 63–78, 2021.
- [8] J. Li, Y. Ren, S. Fang, K. Li, and M. Sun, "Federated Learning-Based Ultra-Short Term Load Forecasting in Power Internet of Things," in Proceedings of the IEEE International Conference on Energy Internet (ICEI), Sydney, NSW, Australia, 24–28 August 2020, pp. 63–68.
- [9] A. Agarwal, B. Xie, I. Vovsha, O. Rambow, and R. Passonneau, "Sentiment Analysis of Twitter Data," in Proceedings of the Workshop on Languages in Social Media, 2011, pp. 30-38.
- [10] B. Liu, "Sentiment Analysis and Opinion Mining," Synthesis Lectures on Human Language Technologies, vol. 5, no. 1, pp. 1-167, 2012.

- [11] E. Cambria, "Affective Computing and Sentiment Analysis," IEEE Intelligent Systems, vol. 31, no. 2, pp. 102-107, 2016.
- [12] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs Up? Sentiment Classification Using Machine Learning Techniques," in Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2002, pp. 79-86, doi: 10.3115/1118693.1118704.
- [13] A. M. Ramadhani and H. S. Goo, "Twitter sentiment analysis using deep learning methods," in Proc. 7th International Annual Engineering Seminar (InAES), Yogyakarta, Indonesia, 2017.
- [14] T. Mullen and N. Collier, "Sentiment Analysis Using Support Vector Machines with Diverse Information Sources," in Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2004, pp. 412-418.
- [15] C. Cortes and V. Vapnik, "Support-Vector Networks," Machine Learning, vol. 20, no. 3, pp. 273-297, Sep. 1995, doi: 10.1007/BF00994018.
- [16] R. Moraes, J. F. Valiati, and W. P. Gavi~ao Neto, "Document-Level Sentiment Classification: An Empirical Comparison Between SVM and ANN," Expert Systems with Applications, vol. 40, no. 2, pp. 621-633, Feb. 2013, doi: 10.1016/j.eswa.2012.07.059.
- [17] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning", Nature, vol. 521, pp. 436-444, Macmillan Publishers, May 2015.
- [18] N. binti Abdul Hamid and N. Nur Binti Amir Sjarif, "Handwritten Recognition Using SVM, KNN and Neural Network".
- [19] C.-L. Liu, K. Nakashima, H. Sako, and H. Fujisawa, "Handwritten Digit Recognition: Benchmarking of State-of-the-Art Techniques", Elsevier, Pattern Recognition, vol. 36, pp. 2271–2285, 2003.
- [20] D. Ciresan, U. Meier, and J. Schmidhuber, "Multi-Column Deep Neural Networks for Image Classification," 2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 3642–3649, 2012.
- [21] D.C. Ciresan, A. Giusti, L.M. Gambardella, and J. Schmidhuber, "Mitosis Detection in Breast Cancer Histology Images with Deep Neural Networks," Medical Image Computing and Computer-Assisted Intervention–MICCAI 2013, pp. 411–418, Springer, 2013.

- [22] C. Tsai, K. Chen, Y. Hu, and W. Chen, "Improving text summarization of online hotel reviews with review helpfulness and sentiment," *Tourism Management*, vol. 80, no. 104122, 2020.
- [23] B. Das and S. Chakraborty, "An Improved Text Sentiment Classification Model Using TF-IDF and Next Word Negation," *arXiv preprint arXiv:1806.06407*, cs.CL, 2018.
- [24] X. Li, X. Sun, Z. Xu, and Y. Zhou, "Explainable Sentence-Level Sentiment Analysis for Amazon Product Reviews," *arXiv preprint arXiv:2111.06070*, cs.CL, 2021.
- [25] M. Das, K. Selvakumar, and P. J. A. Alphonse, "A Comparative Study on TF-IDF Feature Weighting Method and its Analysis using Unstructured Dataset," *arXiv preprint arXiv:2308.04037*, cs.CL, 2023.
- [26] B. P. R. Guda, M. Srivastava, and D. Karkhanis, "Sentiment Analysis: Predicting Yelp Scores," *arXiv preprint arXiv:2201.07999*, cs.LG, 2020.
- [27] M. Khan and K. Malik, "Sentiment Classification of Customer's Reviews About Automobiles in Roman Urdu," in *Advances in Intelligent Systems and Computing*, 2018, pp. 630-640.
- [28] M. Anyanwu and S. Shiva, "Comparative Analysis of Serial Decision Tree Classification Algorithms," *International Journal of Computer Science and Security*, vol. 3, no. 3, 2009.
- [29] M. J. Muzammil, S. Qazi, and T. Ali, "Comparative Analysis of Classification Algorithms Performance for a Statistical-Based Intrusion Detection System," 2013 3rd IEEE International Conference on Computer, Control and Communication (IC4), 2013, pp. 1-6, doi: 10.1109/IC4.2013.6653738.
- [30] C. Anuradha and T. Velmurugan, "A Comparative Analysis on the Evaluation of Classification Algorithms in the Prediction of Students' Performance," *Indian Journal of Science and Technology*, vol. 8, pp. 974-6846, 2015, doi: 10.17485/ijst/2015/v8i15/74555.
- [31] M. L. B. Estrada, R. Z. Cabada, R. O. Bustillos, and M. Graff, "Opinion mining and emotion recognition applied to learning environments," *Expert Systems with Applications*, vol. 150, no. 113265, 2020.
- [32] A. Onan, "Sentiment analysis on product reviews based on weighted word embeddings and deep neural networks," *Concurrency Computation: Practice and Experience*, Special Issue, pp. 1-12, 2020.

- [33] S. Alotaibi and A. Al-Rasheed, "A review and comparative analysis of sentiment analysis techniques," *Informatica*, vol. 46, no. 6, July 2022. DOI: 10.31449/inf.v46i6.3991.
- [34] N. Zainuddin and A. Selamat, "Sentiment analysis using Support Vector Machine," 2014 International Conference on Computer, Communications, and Control Technology (I4CT), Langkawi, Malaysia, 2014, pp. 333-337, doi: 10.1109/I4CT.2014.6914200.
- [35] Evgeniou, T., Pontil, M., "Support Vector Machines: Theory and Applications." In: Paliouras, G., Karkaletsis, V., Spyropoulos, C.D. (eds) *Machine Learning and Its Applications. ACAI 1999. Lecture Notes in Computer Science*, vol 2049. Springer, Berlin, Heidelberg.
- [36] F. M. Shiri, T. Perumal, N. Mustapha, and R. Mohamed, "A Comprehensive Overview and Comparative Analysis on Deep Learning Models: CNN, RNN, LSTM, GRU," September 2023, DOI: 10.13140/RG.2.2.11938.81609.
- [37] J. E. T. Akinsola, "Supervised Machine Learning Algorithms: Classification and Comparison," June 2017, DOI: 10.14445/22312803/IJCTT-V48P126.
- [38] S. K. Lakshminarayanan and J. McCrae, "A Comparative Study of SVM and LSTM Deep Learning Algorithms for Stock Market Prediction," National University of Ireland Galway.
- [39] P. Sudhir and V. D. Suresh, "Comparative Study of Various Approaches, Applications, and Classifiers for Sentiment Analysis," RNSIT, Bengaluru, India.